

# PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL

## FORMATO GUÍA DE APRENDIZAJE

### 1. IDENTIFICACIÓN DE LA GUÍA DE APRENDIZAJE

- **Denominación del Programa de Formación:** Fundamentos de Programación en Python.
- **Competencia:** Construcción de algoritmos y lógica computacional básica.
- **Resultados de Aprendizaje:** 1. Comprender la ejecución secuencial de un programa. 2. Almacenar y recuperar datos usando variables. 3. Controlar el flujo del programa mediante condiciones y ciclos. 4. Agrupar información inmutable y mutable (Tuplas y Listas). 5. Etiquetar y estructurar información compleja (Diccionarios).
- **Duración de la Guía:** 60 horas.

### 2. PRESENTACIÓN

¿Qué es programar? Imagina que la computadora es un empleado extremadamente rápido, pero que no sabe hacer absolutamente nada por sí solo. Programar es, simplemente, darle una lista de instrucciones paso a paso para que haga el trabajo por ti.

En esta guía no vamos a memorizar códigos complejos. Vamos a aprender a **hablarle a la máquina**. Empezaremos haciendo que la computadora nos salude, luego le enseñaremos a recordar datos (variables), a tomar decisiones (if) y finalmente a organizar grandes cantidades de información (listas, tuplas y diccionarios).

El objetivo es que al terminar esta guía, dejes de ser un "usuario" de tecnología y te conviertas en un "creador".

### 3. FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

#### FASE 1: El programa habla y recuerda (Print y Variables)

**Concepto:** La computadora lee el código de arriba hacia abajo, línea por línea. `print` sirve para mostrar mensajes en la pantalla. Las **variables** son como cajas con etiquetas donde guardamos datos para usarlos después.

**Explicación Detallada (El Por Qué Técnico):** Cuando creas una variable, Python le pide al Sistema Operativo un pequeño bloque en la memoria RAM de tu computadora. Al ponerle un nombre (ej. `edad`), estás creando un "puntero" o un atajo para que la CPU sepa dónde ir a buscar ese dato en la RAM milisegundos después. *El superpoder de Python:* Es de "tipado

dinámico". A diferencia de otros lenguajes donde debes declarar el tamaño exacto de la caja, en Python la caja se ajusta automáticamente y, si dejas de usarla, un proceso interno llamado *Garbage Collector* (recolector de basura) libera esa memoria RAM automáticamente para que tu programa no colapse el equipo.

### Ejemplo Guiado 1.1: Creando cajas en memoria ( 01\_hola\_memoria.py )

```
# 1. Salida estática: El programa "habla" mostrando texto en pantalla
print("Iniciando el sistema de diagnóstico...")

# 2. Asignación de estado: Guardando valores en la memoria (creando "cajas")
temperatura_cpu = 45          # Guardamos un número entero (Integer)
voltaje = 1.2                 # Guardamos un número decimal (Float)
estado_sistema = "Estable"    # Guardamos texto (String - siempre entre comillas)

# 3. Recuperación e interpolación (La letra 'f' antes de las comillas permite met
print(f"Estado actual: {estado_sistema}")
print(f"Lecturas: {temperatura_cpu}°C a {voltaje}V")

# 4. El flujo termina cuando no hay más líneas de código
print("Diagnóstico finalizado.")
```

### Ejemplo Guiado 1.2: Actualizando las cajas (Matemáticas básicas)

```
# 1. Creamos una caja con nuestro dinero inicial
billetera = 50000
print(f"Dinero inicial: ${billetera}")

# 2. Reasignación: Tomamos lo que había en la billetera, le restamos 15000 y lo g
billetera = billetera - 15000
print(f"Compré un almuerzo. Nuevo saldo: ${billetera}")

# 3. Operaciones múltiples
bonificacion = 20000
billetera = billetera + bonificacion
print(f"Recibí un pago. Saldo final: ${billetera}")
```

### Ejemplo Guiado 1.3: Conociendo y transformando las cajas (Tipos de datos)

```
# A veces necesitamos saber exactamente QUÉ tipo de dato tiene una caja
# Usamos la función type() para que Python nos revele el secreto

dato_desconocido = "100" # ¡Atención! Es un número, pero está entre comillas. Es
print(f"El dato es: {dato_desconocido} y su tipo es: {type(dato_desconocido)}")
```

```
# Conversión (Casting): Transformando la caja de Texto a Número (Integer)
dato_convertido = int(dato_desconocido)
print(f"Ahora el dato es: {dato_convertido} y su tipo es: {type(dato_convertido)}")

# Ahora sí podemos hacer matemáticas
print(f"Suma matemática real: {dato_convertido + 50}")
```

## Ejercicios Prácticos Fase 1:

1. Crea un archivo llamado `perfil.py` .
2. Crea tres variables: `nombre` (con tu nombre), `edad` (con tu edad en números) y `comida_favorita` (texto).
3. Usa un `print` con la letra `f` para que la consola muestre: *"Hola, me llamo {nombre}, tengo {edad} años y me encanta comer {comida\_favorita}."*
4. **Reto matemático:** Crea una variable `año_nacimiento` (ej. 2000) y `año_actual` (ej. 2026). Crea una tercera variable llamada `edad_calculada` que reste el actual menos el de nacimiento, e imprímela. (Nota: Comprender la diferencia entre el tipo 'String' y el tipo 'Integer' aquí será clave para tu Evidencia de Conocimiento).

## FASE 2: El programa escucha y toma decisiones (Input e If)

**Concepto:** Un programa estático es aburrido. Necesitamos que el usuario ingrese datos (`input` ). Luego, la computadora debe tomar distintos caminos dependiendo de esos datos (`if` = si pasa esto, `elif` = o si pasa esto, `else` = si no, haz esto otro).

**Explicación Detallada (El Por Qué Técnico):** La CPU de tu computadora toma decisiones usando "álgebra booleana". Cualquier condición (ej. `edad >= 18` ) se resuelve internamente en un estado eléctrico: `True` (hay corriente / 1) o `False` (no hay corriente / 0). *El superpoder de `elif`:* Usar `elif` en lugar de muchos `if` separados se llama "evaluación de cortocircuito". Si el primer `if` es verdadero, Python ignora por completo el resto de las comprobaciones. Esto le ahorra ciclos de procesamiento a la CPU y hace que el software sea inmensamente más rápido.

### Ejemplo Guiado 2.1: Decisiones simples ( `02_decisiones.py` )

```
# 1. Entrada de datos: El programa se pausa y espera a que el usuario escriba algo
# Nota: input() siempre guarda texto. Si queremos números, lo envolvemos en int()
edad_usuario = int(input("Por favor, ingresa tu edad en números: "))

# 2. Evaluación lógica: El programa analiza el dato ingresado
print("Evaluando permisos de acceso...")

# 3. Bifurcación (If / Elif / Else): El código toma un solo camino
if edad_usuario >= 18:
    # Si la condición es VERDADERA, entra a este bloque (nota el espacio a la izc
```

```

    print("✅ Acceso concedido: Eres mayor de edad.")

elif edad_usuario >= 13:
    # Si la de arriba fue falsa, pero esta es VERDADERA, entra aquí (Cortocircuit
    print("⚠️ Acceso restringido: Eres adolescente, necesitas permiso de un tutor

else:
    # Si NINGUNA de las anteriores fue verdadera, por descarte entra aquí
    print("❌ Acceso denegado: Eres menor de edad.")

# 4. Esta línea no tiene espacios a la izquierda, así que se ejecuta SIEMPRE
print("Gracias por usar el sistema.")

```

## Ejemplo Guiado 2.2: Decisiones Múltiples con "Y" / "O" ( and / or )

```

# 1. Pedimos dos datos diferentes al usuario
tiene_licencia = input("¿Tienes licencia de conducir? (si/no): ")
esta_sobrio = input("¿Has bebido alcohol hoy? (si/no): ")

# 2. Usamos 'and' (Y). AMBAS condiciones deben ser verdad para entrar
if tiene_licencia == "si" and esta_sobrio == "no":
    print("🚗 Puedes conducir el vehículo.")
else:
    print("🛑 Entregue las llaves. No puede conducir.")

```

## Ejemplo Guiado 2.3: Decisiones Anidadas (El laberinto lógico)

```

# A veces, necesitamos tomar una decisión DENTRO de otra decisión.
print("--- Diagnóstico de Red ---")
hay_internet = input("¿El módem tiene luces encendidas? (si/no): ")

if hay_internet == "si":
    print("Paso 1: El equipo recibe energía.")

    # Este IF está DENTRO del IF principal. Solo se evalúa si el primero fue "si"
    luz_roja = input("¿Alguna de las luces es color ROJO? (si/no): ")
    if luz_roja == "si":
        print("Fallo detectado: Problema en la fibra óptica. Llame a soporte.")
    else:
        print("Todo normal: Su conexión está operando al 100%.")
else:
    print("Fallo crítico: Verifique que el equipo esté conectado a la corriente.")

```

## Ejercicios Prácticos Fase 2:

1. Escribe un programa que pregunte la "contraseña". Si el usuario escribe "secreto123", imprime "Bienvenido al sistema". Si escribe otra cosa, imprime "Intruso detectado".
2. Escribe un programa que pregunte la nota de un examen (de 0 a 100). Si es mayor o igual a 60, imprime "Aprobado". Si es menor, "Reprobado".
3. 🌟 [EVIDENCIA DE DESEMPEÑO 1] - Reto Lógico de la Montaña Rusa: Escribe un script llamado `02_reto_logico.py`. Un parque de diversiones solo deja subir a la montaña rusa si el usuario mide **más de 150 cm** Y tiene **más de 12 años**. Pide ambos datos por teclado usando `input()` (recuerda convertirlos a `int()` o `float()`). Utiliza un condicional `if` con el operador `and` para imprimir si tiene permitido o denegado el acceso. *El instructor validará en vivo tu capacidad para indentar correctamente y construir la lógica.*

### FASE 3: El programa repite sin cansarse (Ciclo For)

**Concepto:** Si te pido que cuentes hasta 1000, te cansarás. La computadora no. El ciclo `for` (para) nos permite repetir una acción una cantidad específica de veces sin escribir la misma línea de código una y otra vez.

**Explicación Detallada (El Por Qué Técnico):** En programación, el principio fundamental es "DRY" (Don't Repeat Yourself - No te repitas). Si copias y pegas líneas de código, multiplicas la posibilidad de errores y gastas memoria en disco. El ciclo `for` es una instrucción a nivel de procesador que le ordena al "puntero de ejecución" saltar de regreso hacia arriba en el código, hasta que una colección de datos se termine de recorrer. *Precaución:* Los procesadores son tan rápidos que, si les damos una colección gigantesca, podemos consumir toda la capacidad de la CPU.

#### Ejemplo Guiado 3.1: Contando automáticamente ( `03_repeticion.py` )

```
# 1. Bucle For: Automatizando la repetición
# range(1, 6) genera los números del 1 al 5 (el último número no se incluye)
print("Iniciando conteo automatizado:")

for numero in range(1, 6):
    # La variable 'numero' cambia su valor automáticamente en cada vuelta
    print(f"Paso actual: {numero}")

# 2. Iterando sobre un texto (cada letra es un paso)
palabra_clave = "SENA"
print(f"Deletreando la palabra {palabra_clave}:")

for letra in palabra_clave:
    # Imprime una letra por cada vuelta del bucle
    print(f"Letra detectada: {letra}")
```

### Ejemplo Guiado 3.2: La Caja Acumuladora (Sumando en cada vuelta)

```
# 1. Creamos una variable en cero ANTES de empezar a repetir
total_ahorrado = 0

# 2. Repetiremos el proceso 3 veces (mes 1, mes 2, mes 3)
for mes in range(1, 4):
    # 3. En cada vuelta, le pedimos dinero al usuario
    consignacion = int(input(f"¿Cuánto dinero vas a ahorrar en el mes {mes}?: "))

    # 4. Sumamos lo que ingresó a nuestra caja acumuladora
    total_ahorrado = total_ahorrado + consignacion

# 5. El total se imprime FUERA del bucle (sin indentación) al terminar todas las
print(f"Ahorro completado! Tienes un total de: ${total_ahorrado}")
```

### Ejemplo Guiado 3.3: Mezclando For e If (El filtro inteligente)

```
# El verdadero poder está en combinar todo lo que hemos aprendido.
print("--- Filtro de Seguridad ---")
edades_acceso = [15, 22, 17, 30, 14]

for edad in edades_acceso:
    # En cada vuelta, evaluamos a la persona actual
    if edad >= 18:
        print(f"✅ Persona de {edad} años: Acceso PERMITIDO.")
    else:
        print(f"❌ Persona de {edad} años: Bloqueada.")
```

### Ejercicios Prácticos Fase 3:

1. Crea un bucle `for` que imprima la tabla de multiplicar del 5 (del 5x1 al 5x10). *Pista: usa variables y multiplicaciones dentro del print.*
2. Haz un programa que cuente del 10 al 1 hacia atrás. *Pista: investiga cómo funciona `range(10, 0, -1)`.*
3. 🧠 [EVIDENCIA DE DESEMPEÑO 2] - Reto Acumulador de Caja Registradora: Escribe un script llamado `03_reto_acumulador.py`. Simula una caja registradora. Crea una variable acumuladora en 0. Usa un bucle `for` que se repita exactamente 5 veces. En cada vuelta, pide al usuario el precio de un producto. Al finalizar el bucle, imprime el costo total de la compra. *El instructor observará tu capacidad para mantener variables fuera y dentro de un bucle sin perder el valor acumulado.*

### FASE 4: Guardando múltiples cosas que pueden cambiar (Listas)

**Concepto:** Hasta ahora, una variable guarda UNA sola cosa. ¿Qué pasa si quiero guardar 100 nombres? Usamos una **Lista**. Es como un archivero con muchos cajones, donde podemos meter, sacar o cambiar cosas libremente.

**Explicación Detallada (El Por Qué Técnico):** Una variable simple reserva un espacio pequeño en la RAM. Una Lista reserva un bloque continuo mucho más grande en la RAM, y le asigna un "índice numérico" a cada casilla (offset). *¿Por qué empezamos a contar desde CERO?* Porque en lenguaje máquina, el "0" indica la distancia exacta desde el inicio del bloque de memoria. Si la lista empieza en la posición 1000 de la RAM, el elemento 0 está a una distancia de "0", o sea, en la posición 1000. Las listas son "mutables": si haces `.append()` y se quedan sin espacio en ese bloque de RAM, Python internamente busca un bloque nuevo más grande, copia todo, y borra el viejo en fracciones de segundo.

#### Ejemplo Guiado 4.1: Creando y leyendo el archivero ( 04\_listas.py )

```
# 1. Creación de una lista: Se usan corchetes [] y se separan por comas
inventario_frutas = ["Manzana", "Pera", "Mango", "Banano"]

# 2. Accediendo a un elemento específico: ¡Los programadores empezamos a contar c
print(f"La primera fruta es: {inventario_frutas[0]}") # Imprime Manzana
print(f"La tercera fruta es: {inventario_frutas[2]}") # Imprime Mango

# 3. La verdadera magia: Combinando Listas y el ciclo For
print("--- Imprimiendo viñetas automáticas ---")

# Por cada 'fruta' individual dentro de la lista 'inventario_frutas', haz lo sigu
for fruta in inventario_frutas:
    print(f"- {fruta} disponible en bodega.")
```

#### Ejemplo Guiado 4.2: Modificando el archivero (Agregar y Quitar)

```
carrito_compras = ["Leche", "Huevos", "Pan"]
print(f"Carrito inicial: {carrito_compras}")

# 1. Agregar al final (.append)
carrito_compras.append("Café")
print(f"Agregué Café: {carrito_compras}")

# 2. Eliminar un elemento específico (.remove)
carrito_compras.remove("Leche")
print(f"Quité la Leche: {carrito_compras}")

# 3. Cambiar un elemento sabiendo su posición (Cajón 0)
carrito_compras[0] = "Huevos Campesinos"
```

```
print(f"Cambié los huevos normales: {carrito_compras}")
```

### Ejercicios Prácticos Fase 4:

1. Crea una lista vacía llamada `mis_amigos = []`.
2. Usa `input` tres veces para pedir el nombre de tres amigos y usa `.append()` para meterlos en la lista.
3. Usa un `for` para imprimir un saludo personalizado a cada amigo de la lista.
4. **Reto Inventario:** Tienes la lista `colores = ["Rojo", "Azul", "Verde"]`. Pídele al usuario que escriba un color que no le guste y bórralo usando `.remove()`. Imprime la lista final.

### FASE 5: Cajas fuertes de seguridad (Tuplas)

**Concepto:** Las listas (Fase 4) son geniales porque cambian. Pero a veces, en un programa, tenemos datos que **nunca deberían cambiar por accidente**, como las coordenadas GPS de una sucursal o los meses del año. Para eso usamos las **Tuplas**. Son listas "solo de lectura".

**Explicación Detallada (El Por Qué Técnico):** Como una lista puede cambiar de tamaño en cualquier momento, Python debe reservar memoria RAM adicional "por si acaso". Las tuplas, en cambio, se sellan en el momento en que se crean ("Inmutabilidad"). Al tener un tamaño fijo exacto, a nivel de hardware la memoria RAM las lee más rápido, pesan menos bytes, y previenen "Bugs" (errores) críticos evitando que otro pedazo de código modifique datos sagrados.

### Ejemplo Guiado 5.1: Creación de Tuplas ( 05\_tuplas.py )

```
# 1. Creación de una tupla: A diferencia de las listas, se usan PARÉNTESIS ()
coordenadas_bogota = (4.7110, -74.0721)
meses_año = ("Enero", "Febrero", "Marzo")

# 2. Se accede exactamente igual que a una lista (empezando desde el cero)
print(f"La latitud de Bogotá es: {coordenadas_bogota[0]}")
print(f"El primer mes es: {meses_año[0]}")

# 3. LO QUE NO SE PUEDE HACER (Inmutabilidad)
# Si intentas: coordenadas_bogota[0] = 5.0 -> ERROR CRÍTICO.
```

### Ejercicios Prácticos Fase 5:

1. Crea una tupla llamada `dias_semana` con los días de lunes a domingo. Pídele al usuario un número del 1 al 7 e imprime el día correspondiente. (*Recuerda la base cero*).
2. Intenta usar `.append()` en tu tupla de `dias_semana` y observa el error rojo que te arroja Python. (*Saber por qué ocurre este error es fundamental para la Evidencia de*

Conocimiento).

## FASE 6: Etiquetas y Valores (Diccionarios)

**Concepto:** Las listas y tuplas guardan cosas en orden (cajón 0, cajón 1). Los **Diccionarios** nos permiten ponerle una "etiqueta" (Clave) a cada cajón (Valor). Son la base del formato JSON, el estándar mundial de transferencia de datos en la web.

**Explicación Detallada (El Por Qué Técnico):** A nivel arquitectónico, un Diccionario funciona usando criptografía matemática ligera (Tablas Hash). En un diccionario, la "Etiqueta" (ej. "nombre") pasa por una fórmula matemática que le dice a la CPU *exactamente* la coordenada en la RAM donde está el dato. El tiempo de búsqueda es instantáneo ( $O(1)$ ).

### Ejemplo Guiado 6.1: Creando un perfil ( 06\_diccionarios.py )

```
# 1. Creación de un diccionario: Se usan llaves {} y el formato "Clave": "Valor"
aprendiz = {
    "nombre": "Carlos",
    "edad": 25,
    "curso": "Programación",
    "activo": True
}

# 2. Accediendo a datos específicos usando la "Etiqueta" (Clave)
print(f"Nombre: {aprendiz['nombre']}")

# 3. Modificando un valor existente y agregando nuevos
aprendiz["edad"] = 26
aprendiz["ciudad"] = "Bogotá" # Nueva clave (Python la crea automáticamente si no
```

### Ejemplo Guiado 6.2: El archivero de diccionarios (Listas + Diccionarios)

```
# 1. Podemos meter diccionarios DENTRO de una lista. Así funcionan las Bases de D
estudiantes = [
    {"nombre": "Ana", "nota": 90},
    {"nombre": "Luis", "nota": 50}
]

# 2. Evaluando a todos los estudiantes de forma automática
for persona in estudiantes:
    if persona["nota"] >= 60:
        print(f"✅ {persona['nombre']} ha APROBADO")
```

## Ejercicios Prácticos Fase 6:

1. Crea un diccionario llamado `mi_mascota` que tenga las llaves: `"nombre"` , `"animal"` , y `"edad"` . Imprímelo.
2. 🌟 [EVIDENCIA DE PRODUCTO FINAL] - Reto Integrador "Concesionario Web": Crea un script llamado `04_producto_concesionario.py` . Tu objetivo es simular la base de datos de un concesionario usando todo lo aprendido.
  - Crea una **lista** vacía llamada `concesionario` .
  - Crea un bucle `for` que se repita exactamente 3 veces.
  - En cada vuelta del bucle, pide al usuario mediante `input()` la "marca", el "modelo" y el "precio" de un carro.
  - Empaqueta esos 3 datos en un **Diccionario** temporal.
  - Usa el método `.append()` para guardar ese diccionario dentro de la lista `concesionario` .
  - Al salir del bucle, crea un segundo bucle `for` que recorra la lista `concesionario` e imprima un informe limpio, ejemplo: *"Vehículo registrado: Marca Toyota, Modelo Corolla, Precio: \$50000"*.
  - **Entrega:** Este archivo debe estar perfectamente documentado con comentarios ( `#` ) explicando cada paso y debe subirse al LMS Zajuna comprimido en un archivo `.zip` .

#### 4. PREPARACIÓN PARA LA EVIDENCIA DE CONOCIMIENTO

Antes de presentar el cuestionario en el LMS, asegúrate de dominar conceptualmente los siguientes puntos extraídos directamente de esta guía:

1. **Tipos de Datos en RAM:** Diferenciar cuándo un dato es un `String` (texto en comillas), un `Integer` (entero matemático), un `Float` (decimal) o un `Boolean` (`True/False`).
2. **Mutabilidad vs Inmutabilidad:** Explicar técnicamente por qué una Lista ( `[]` ) puede crecer o encogerse en tiempo de ejecución (memoria dinámica), mientras que una Tupla ( `()` ) bloquea sus datos y arroja un error si se intenta modificar (protección de estado).
3. **Lógica Booleana:** Comprender cómo el operador lógico `and` exige que *todas* las condiciones sean verdaderas, mientras que `or` exige que *al menos una* lo sea. Entender la ventaja de procesamiento de usar `elif` (cortocircuito) en lugar de múltiples `if` .

#### 5. GLOSARIO DE TÉRMINOS PARA PRINCIPIANTES

- **Variable / Puntero:** Una referencia lógica a un bloque de la memoria RAM de la computadora donde guardamos un dato temporal.
- **String / Integer / Float / Boolean:** Tipos de datos primitivos en programación.

- **Indentación:** Los espacios en blanco al inicio de una línea de código. En Python, agrupan qué líneas pertenecen a un bloque `if` o a un `for` (Reemplaza las llaves `{ }` de otros lenguajes).
- **Inmutabilidad:** Característica fundamental de las Tuplas. Significa que una vez el bloque de memoria es asignado, su contenido no puede ser modificado (Read-Only).
- **JSON (JavaScript Object Notation):** Formato universal de intercambio de datos en la web. En Python, su estructura es exactamente equivalente a los Diccionarios.

## 6. PLANTEAMIENTO DE EVIDENCIAS DE APRENDIZAJE PARA LA EVALUACIÓN

*Nota: La evaluación se divide según el modelo pedagógico del SENA estructurado en Conocimiento, Desempeño y Producto.*

| Fase del proyecto formativo | Actividad del proyecto formativo                                                                    | Actividad de Aprendizaje                                                          | Evidencias de Aprendizaje                                                                                                                                                                            | Criterios de Evaluación                                                                                                                                                                                | Técnicas e Instrumentos de Evaluación                       |
|-----------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| Análisis / Planeación       | Determinar los requerimientos lógicos e interfaces para la construcción del sistema de información. | AP01 - Apropiar fundamentos algorítmicos.                                         | 1. Evidencia de Conocimiento:                                                                                                                                                                        | <ul style="list-style-type: none"> <li>• Identifica los tipos de datos en la memoria RAM.</li> </ul>                                                                                                   | Técnica: Formulación de preguntas.                          |
|                             |                                                                                                     | Asimilar la ejecución secuencial, variables, tipos de datos y operadores lógicos. | <p>Cuestionario técnico y conceptual sobre tipos de datos en RAM, diferencias entre estructuras mutables (Listas) e inmutables (Tuplas), y evaluación de expresiones booleanas en condicionales.</p> | <ul style="list-style-type: none"> <li>• Diferencia correctamente entre una tupla y una lista, justificando su inmutabilidad.</li> <li>• Reconoce la sintaxis correcta del lenguaje Python.</li> </ul> | Instrumento: Cuestionario (LMS Zajuna).                     |
| Análisis / Planeación       | Determinar los requerimientos lógicos e interfaces para la construcción del sistema de información. | AP02 - Aplicar control de flujo y estructuras de iteración.                       | 2. Evidencia de Desempeño:                                                                                                                                                                           | <ul style="list-style-type: none"> <li>• Controla el flujo del programa utilizando sentencias de repetición y decisión, de acuerdo con la</li> </ul>                                                   | Técnica: Observación Sistemática y Resolución de Problemas. |
|                             |                                                                                                     |                                                                                   | Participación directa y                                                                                                                                                                              |                                                                                                                                                                                                        |                                                             |

|                              |                                                                                                     |                                                                                                                |                                                                                                                                                                                 |                                                                                                                                                  |                                                                                                                |
|------------------------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
|                              |                                                                                                     | Desarrollar y validar algoritmos base, utilizando condicionales y ciclos en lenguaje Python.                   | observación del aprendiz codificando en vivo el <b>Reto Acumulador de Caja Registradora</b> (Fase 3 - Ciclos For) y el <b>Reto Lógico de la Montaña Rusa</b> (Fase 2 - If/And). | lógica algorítmica.<br><br>• Soluciona problemas básicos de sintaxis e indentación de manera autónoma frente a la consola.                       | <b>Instrumento:</b><br>Rúbrica de Desempeño / Lista de Chequeo.                                                |
| <b>Análisis / Planeación</b> | Determinar los requerimientos lógicos e interfaces para la construcción del sistema de información. | <b>AP03 - Implementar estructuras de datos (Listas y Diccionarios) para el manejo de información temporal.</b> | <b>3. Evidencia de Producto:</b><br><br>Entrega de archivo comprimido (.zip) alojado en LMS Zajuna que contenga el                                                              | • Almacena y estructura la información temporal en memoria RAM de forma eficiente, simulando modelos de bases de datos mediante el uso de ... .. | <b>Técnica:</b><br>Valoración de Producto.<br><br><b>Instrumento:</b><br>Lista de Verificación de Código (Code |